

Software Metrics for Project Management

Module - III

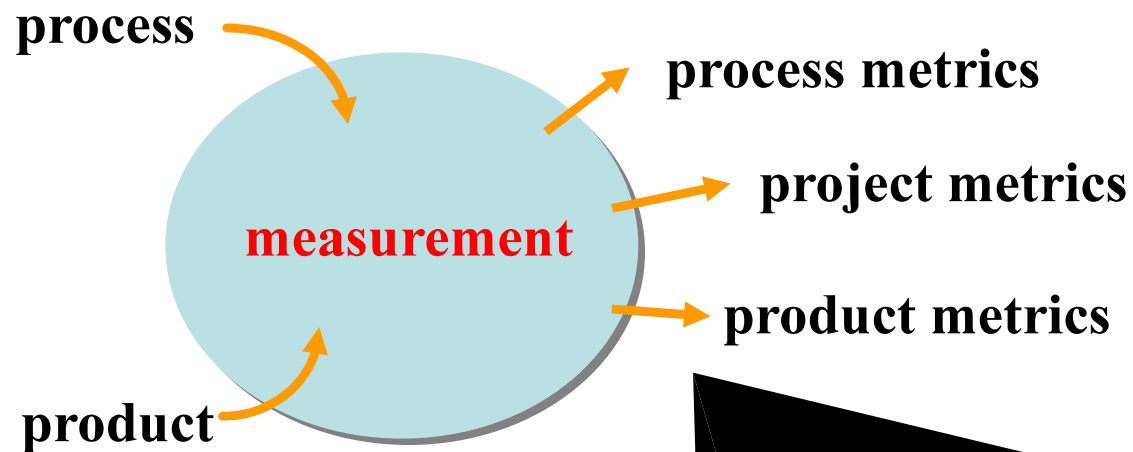
Terminology

- Measure: **Quantitative indication** of the extent, amount, dimension, or size of some attribute of a product or process. A single data point
- Metrics: The degree to which a system, component, or process possesses a given attribute. Relates several measures (e.g. average number of errors found per person hour)
- Indicators: A combination of metrics that provides insight into the software process, project or product
- Direct Metrics: Immediately measurable attributes (e.g. line of code, execution speed, defects reported)
- Indirect Metrics: Aspects that are not immediately quantifiable (e.g. functionality, quantity, reliability)

Terminology

- Faults:
 - Errors: An **error** is a human action that produces an incorrect result.
 - Defects: is an imperfection in a software product.
 - Bug: A "fault" becomes a "bug" once it is officially identified and logged in a tracking system.
 - Failure: the inability of a software system or component to perform its required functions within specified performance requirements.

A Good Manager Measures

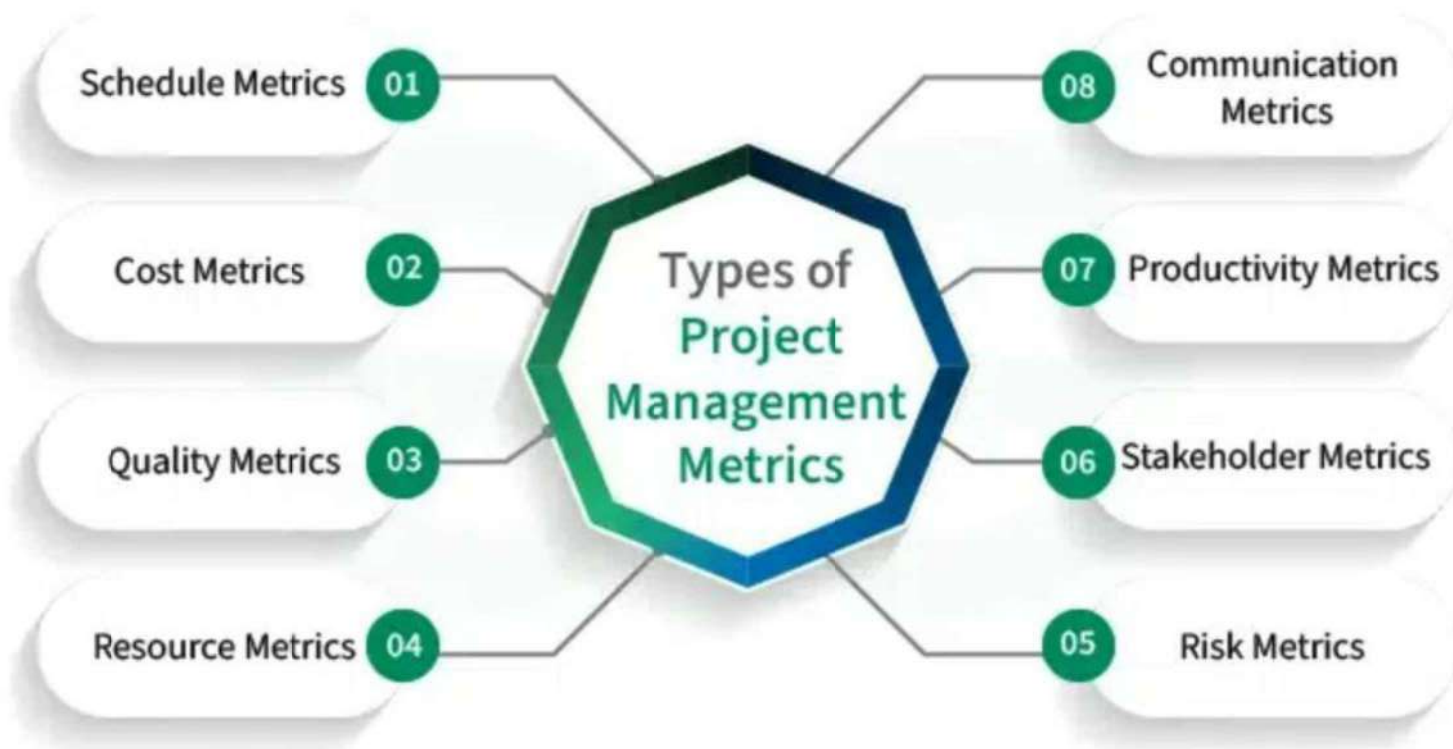


“Not everything that can be counted counts, and not everything that counts can be counted.” - Einstein

**What do we
use as a
basis?**

- size?
- function?

Types of Project Management Metrics



Metric Category	Classification	Rationale
Schedule Metrics	Project Metrics	Track adherence to timelines and milestones, reflecting project-level success.
Cost Metrics	Project Metrics	Measure budget performance and financial health of the project.
Quality Metrics	Product Metrics	Assess deliverable quality (defects, compliance, performance).
Resource Metrics	Process Metrics	Evaluate efficiency of resource allocation and utilization.
Risk Metrics	Project Metrics	Monitor project-level uncertainties and mitigation effectiveness.
Stakeholder Metrics	Project Metrics	Capture stakeholder satisfaction and engagement, tied to project outcomes.
Productivity Metrics	Process Metrics	Reflect efficiency of team processes and throughput.
Communication Metrics	Process Metrics	Assess effectiveness of information flow and collaboration processes.

Process Metrics

- Focus on quality achieved as a consequence of a repeatable or managed process. Strategic and Long Term.
 - Statistical Software Process Improvement (SSPI). Error Categorization and Analysis:
 - All errors and defects are categorized by origin
 - The cost to correct each error and defect is recorded
 - The number of errors and defects in each category is computed
 - Data is analyzed to find categories that result in the highest cost to the organization
 - Plans are developed to modify the process
 - Defect Removal Efficiency (DRE). Relationship between errors (E) and defects (D). The ideal is a DRE of 1:

$$DRE = E / (E + D)$$
- .

Productivity & Output Metrics

- Velocity: (Agile) The amount of work (usually in story points) a team completes during a single sprint.
- Throughput: The total number of tasks or tickets completed within a specific timeframe (e.g., tasks per week).
- Cycle Time: The time it takes for an individual task to move from "In Progress" to "Done."

Utilization & Capacity Metrics

- Resource Utilization Rate: The percentage of a team member's total available time spent on billable or project-related tasks.

$$\text{Utilization Rate} = \left(\frac{\text{Actual Hours Worked}}{\text{Total Available Hours}} \right) \times 100$$

- Capacity: The total amount of work a team can realistically handle in a period, accounting for vacations, meetings, and administrative tasks.
- Resource Allocation: A breakdown of how many people are assigned to specific phases (e.g., 3 developers on Frontend, 2 on QA).

Financial & Cost Metrics

- Cost Variance (CV): The difference between the budgeted cost of work performed and the actual cost.
- Burn Rate: The rate at which the project budget is being consumed, usually measured monthly.
- Planned Value (PV) vs. Actual Cost (AC): Comparing the value of the work intended to be finished by a certain date against what was actually spent.

Quality & Stability Metrics

- Defect Density: The number of confirmed bugs found in a software component per unit of size (like 1,000 lines of code).
- Lead Time: The total time from when a requirement is logged until it is delivered to the user.
- Resource Turnover Rate: How frequently team members leave the project, which can significantly impact knowledge retention and "ramp-up" costs.

Productivity Metrics

Metric	What It Measures	Why It Matters
Velocity	Amount of work completed per iteration (often in story points).	Indicates team capacity and helps forecast future sprints.
Cycle Time	Time taken to complete a task from start to finish.	Shorter cycle times reflect faster delivery and reduced bottlenecks.
Lead Time	Time from request initiation to delivery.	Captures responsiveness to customer needs.
Throughput	Number of tasks or features completed in a given period.	Shows overall team productivity and workload handling.
Defect Density	Number of defects per unit of code (e.g., per KLOC).	Balances productivity with quality, ensuring speed doesn't compromise reliability.
Code Churn	Frequency of code changes or rewrites.	High churn may indicate inefficiency or unclear requirements.

Productivity Metrics

Commit-to-Deploy Ratio	Time between code commit and deployment.	Reflects efficiency of CI/CD pipelines and automation.
Team Utilization Rate	Percentage of time team members spend on productive work vs. overhead.	Helps identify resource allocation issues.
Cumulative Flow	Visualization of work items across stages (to-do, in progress, done).	Highlights bottlenecks and process inefficiencies.
Sprint Burndown	Rate at which tasks are completed during a sprint.	Tracks progress against planned work and identifies delays.

Communication Metrics

Metric	What It Measures	Why It Matters
Message Clarity Index	Percentage of project communications that are clear, concise, and actionable.	Prevents misinterpretation and rework.
Communication Frequency	Number of updates, meetings, or reports shared per week.	Ensures stakeholders are consistently informed.
Response Time	Average time taken to reply to project-related queries.	Reflects team agility and reduces bottlenecks.
Stakeholder Engagement Rate	Percentage of stakeholders actively participating in meetings or feedback sessions.	Indicates alignment and buy-in from stakeholders.
Meeting Effectiveness Score	Ratio of actionable outcomes to total meetings held.	Tracks whether meetings drive progress or waste time.
Information Accessibility	Percentage of project documents and updates available in shared repositories.	Ensures transparency and reduces dependency on individuals.

Communication Metrics

Collaboration Tool Usage	Frequency of use of platforms like Slack, Teams, or Jira for project communication.	Shows adoption of digital communication channels.
Escalation Rate	Number of issues escalated due to poor communication.	Identifies gaps in clarity or responsiveness.
Feedback Loop Completion	Percentage of feedback items addressed and closed within agreed timelines.	Demonstrates responsiveness and continuous improvement.
Communication Satisfaction Score	Survey-based metric on how well team members and stakeholders perceive communication.	Provides qualitative insight into communication health.

Project Metrics

- Used by a project manager and software team to adapt project workflow and technical activities. Tactical and Short Term.
- Purpose:
 - Minimize the development schedule by making the necessary adjustments to avoid delays and mitigate problems
 - Assess product quality on an ongoing basis
- Metrics:
 - Effort or time per SE task;
 - Errors uncovered per review hour
 - Scheduled vs. actual milestone dates;
 - Number of changes and their characteristics
 - Distribution of effort on SE tasks

Schedule Metrics

Metric	What It Measures	Why It Matters
Planned Value (PV)	The budgeted cost of work scheduled at a given time.	Establishes baseline expectations for schedule progress.
Earned Value (EV)	The budgeted cost of work actually performed.	Shows how much scheduled work has been completed.
Schedule Variance (SV = EV – PV)	Difference between earned value and planned value.	Positive SV means ahead of schedule; negative means behind.
Schedule Performance Index (SPI = EV ÷ PV)	Ratio of earned value to planned value.	SPI < 1 indicates delays; SPI > 1 indicates efficiency.
On-Time Completion Rate	Percentage of tasks or milestones completed by their planned deadlines.	Direct measure of schedule adherence.

Schedule Metrics

Milestone Achievement Rate	Number of milestones achieved on time vs. planned.	Tracks critical project checkpoints.
Critical Path Variance	Deviations in tasks on the critical path.	Identifies risks that directly affect project completion date.
Cycle Time	Time taken to complete a task or feature.	Helps identify bottlenecks and improve scheduling accuracy.
Lead Time	Time from request initiation to delivery.	Reflects responsiveness and overall schedule efficiency.
Average Task Delay	Mean deviation between planned and actual task completion dates.	Highlights recurring schedule slippage.

Cost Metrics

Metric	What It Measures	Why It Matters
Planned Value (PV)	Budgeted cost of work scheduled at a given time.	Establishes baseline financial expectations.
Earned Value (EV)	Budgeted cost of work actually performed.	Shows financial progress relative to plan.
Actual Cost (AC)	Real cost incurred for completed work.	Provides the true expenditure picture.
Cost Variance (CV = EV – AC)	Difference between earned value and actual cost.	Positive CV means under budget; negative means over budget.
Cost Performance Index (CPI = EV ÷ AC)	Efficiency of cost utilization.	CPI < 1 indicates overspending; CPI > 1 indicates cost efficiency.

Cost Metrics

Budget at Completion (BAC)	Total planned budget for the project.	Defines financial scope and limits.
Estimate at Completion (EAC)	Forecasted total cost based on current performance.	Helps predict final expenditure.
Estimate to Complete (ETC)	Expected cost to finish remaining work.	Guides resource allocation and corrective actions.
Variance at Completion (VAC = BAC – EAC)	Difference between planned and forecasted total cost.	Shows expected budget surplus or deficit.
Return on Investment (ROI)	Ratio of project benefits to costs.	Evaluates financial justification of the project.
Cost of Quality (CoQ)	Cost of prevention, appraisal, and fixing defects.	Balances spending on quality assurance vs. rework.
Resource Cost Rate	Cost per resource unit (e.g., developer per hour).	Helps optimize staffing and outsourcing decisions.

Risk Metrics

Metric	What It Measures	Why It Matters
Risk Exposure	Potential impact $\times$ probability of risks.	Quantifies overall project vulnerability.
Risk Severity Index	Weighted score of risk likelihood and impact.	Helps prioritize critical risks.
Risk Occurrence Rate	Frequency of risks materializing during the project.	Tracks effectiveness of risk prevention.
Risk Mitigation Effectiveness	Percentage of identified risks successfully mitigated.	Shows how well risk strategies work.
Open vs. Closed Risks	Ratio of unresolved risks to resolved ones.	Indicates risk management progress.
Residual Risk Level	Remaining risk after mitigation actions.	Ensures awareness of unavoidable risks.
Risk Response Time	Average time taken to address identified risks.	Reflects agility in risk handling.
Cost of Risk Management	Resources spent on risk prevention and mitigation.	Balances proactive investment vs. reactive costs.

Stakeholder Metrics

Metric	What It Measures	Why It Matters
Stakeholder Satisfaction Index	Survey-based score of stakeholder happiness with project progress.	Ensures alignment with expectations.
Engagement Rate	Percentage of stakeholders actively participating in meetings, reviews, or feedback.	Reflects involvement and buy-in.
Feedback Responsiveness	Average time to act on stakeholder feedback.	Builds trust and credibility.
Communication Coverage	Percentage of stakeholders receiving timely updates.	Prevents information gaps.
Conflict Resolution Rate	Number of stakeholder conflicts resolved vs. raised.	Tracks collaboration health.

Stakeholder Metrics

Stakeholder Influence Weighting	Prioritization of stakeholders based on power/interest.	Ensures critical voices are heard.
Approval Cycle Time	Time taken for stakeholders to approve deliverables or changes.	Impacts schedule efficiency.
Escalation Frequency	Number of issues escalated by stakeholders.	Indicates dissatisfaction or communication gaps.

Product Metrics

- Focus on the quality of deliverables
- Product metrics are combined across several projects to produce process metrics
- Metrics for the product:
 - Measures of the Analysis Model
 - Complexity of the Design Model
 1. Internal algorithmic complexity
 2. Architectural complexity
 3. Data flow complexity
 - Code metrics

Normalization for Metrics

- How does an organization combine metrics that come from different individuals or projects?
- Depend on the size and complexity of the project
- Normalization: compensate for complexity aspects particular to a product
- Normalization approaches:
 - Size oriented (lines of code approach)
 - Function oriented (function point approach)

Typical Normalized Metrics

Project	LOC	FP	Effort (P/M)	R(000)	Pp. doc	Errors	Defects	People
alpha	12100	189	24	168	365	134	29	3
beta	27200	388	62	440	1224	321	86	5
gamma	20200	631	43	314	1050	256	64	6

- Size-Oriented:
 - errors per KLOC (thousand lines of code), defects per KLOC, R per LOC, page of documentation per KLOC, errors / person-month, LOC per person-month, R / page of documentation
- Function-Oriented:
 - errors per FP, defects per FP, R per FP, pages of documentation per FP, FP per person-month

OO Metrics: Distinguishing Characteristics

- The following characteristics require that special OO metrics be developed:
 - Encapsulation — Concentrate on classes rather than functions
 - Information hiding — An information hiding metric will provide an indication of quality
 - Inheritance — A pivotal indication of complexity
 - Abstraction — Metrics need to measure a class at different levels of abstraction and from different viewpoints
- Conclusion: the class is the fundamental unit of measurement

OO Project Metrics

- Number of Scenario Scripts (Use Cases):
 - Number of use-cases is directly proportional the number of classes needed to meet requirements
 - A strong indicator of program size
- Number of Key Classes (Class Diagram):
 - A key class focuses directly on the problem domain
 - NOT likely to be implemented via reuse
 - Typically 20-40% of all classes are key, the rest support infrastructure (e.g. GUI, communications, databases)
- Number of Subsystems (Package Diagram):
 - Provides insight into resource allocation, scheduling for parallel development and overall integration effort

OO Analysis and Design Metrics

- Related to Analysis and Design Principles
- Complexity:
 - Weighted Methods per Class (WMC): Assume that n methods with cyclomatic complexity are defined for a class C:

$$WMC = \sum c_i \quad c_1, c_2, \dots, c_n$$

- Depth of the Inheritance Tree (DIT): The maximum length from a leaf to the root of the tree. Large DIT leads to greater design complexity but promotes reuse
- Number of Children (NOC): Total number of children for each class. Large NOC may dilute abstraction and increase testing

Further OOA&D Metrics

- Coupling:
 - Coupling between Object Classes (COB): Total number of collaborations listed for each class in CRC cards. Keep COB low because high values complicate modification and testing
 - Response For a Class (RFC): Set of methods potentially executed in response to a message received by a class. High RFC implies test and design complexity
- Cohesion:
 - Lack of Cohesion in Methods (LCOM): Number of methods in a class that access one or more of the same attributes. High LCOM means tightly coupled methods

OO Testability Metrics

- Encapsulation:
 - Percent Public and Protected (PAP): Percentage of attributes that are public. Public attributes can be inherited and accessed externally. High PAP means more side effects
 - Public Access to Data members (PAD): Number of classes that access another classes attributes. Violates encapsulation
- Inheritance:
 - Number of Root Classes (NRC): Count of distinct class hierarchies. Must all be tested separately
 - Fan In (FIN): The number of superclasses associated with a class. $FIN > 1$ indicates multiple inheritance. Must be avoided
 - Number of Children (NOC) and Depth of Inheritance Tree (DIT): Superclasses need to be retested for each subclass

$$DRE = E / (E + D)$$

Quality Metrics

- Measures conformance to explicit requirements, following specified standards, satisfying of implicit requirements
 - Software quality can be difficult to measure and is often highly subjective
1. Correctness:
 - The degree to which a program operates according to specification
 - Metric = Defects per FP
 2. Maintainability:
 - The degree to which a program is amenable to change
 - Metric = Mean Time to Change. Average time taken to analyze, design, implement and distribute a change

Quality Metrics: Further Measures

3. Integrity:

- The degree to which a program is impervious to outside attack

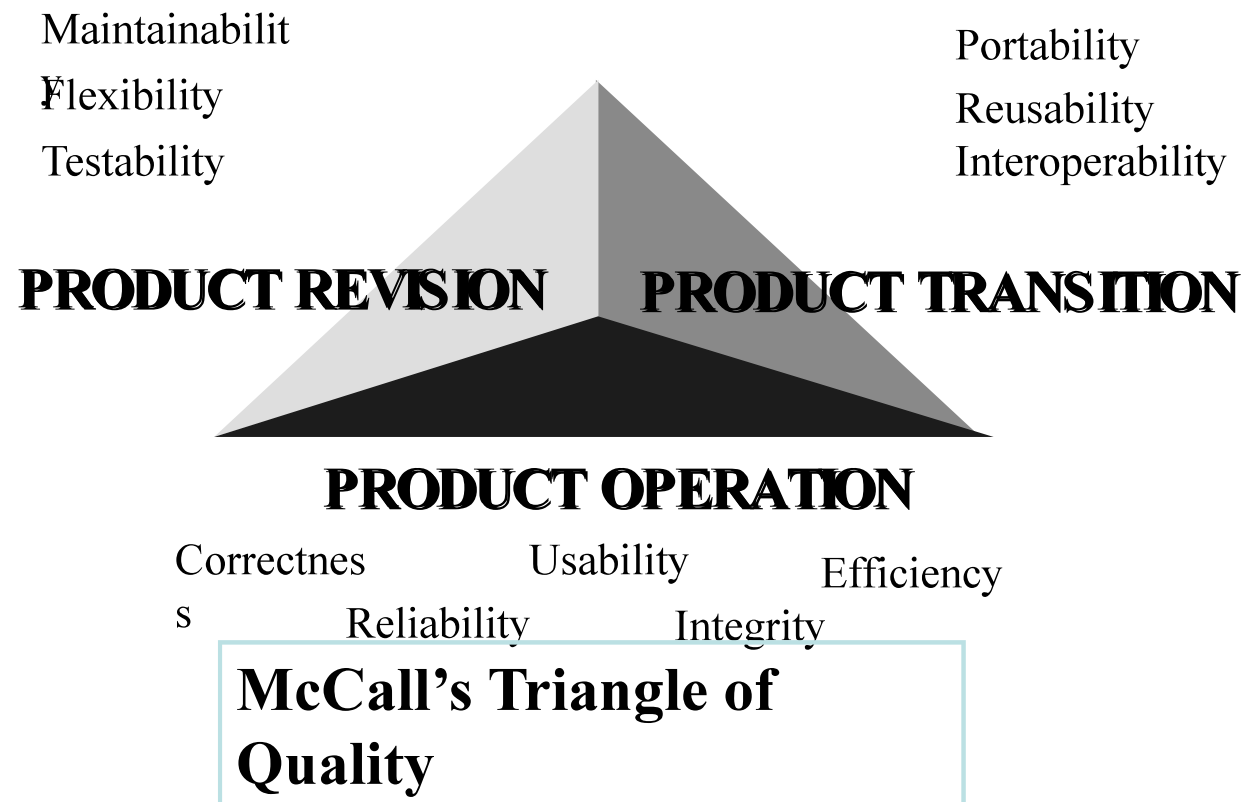
- $$\sum_i (1 - t_i) \cdot s_i$$

- Summed over all types of security attacks, i , where t = threat (probability that an attack of type i will occur within a given time) and s = security (probability that an attack of type i will be repelled)

4. Usability:

- The degree to which a program is easy to use.
- Metric = (1) the skill required to learn the system, (2) the time required to become moderately proficient, (3) the net increase in productivity, (4) assessment of the users attitude to the system

Quality Metrics: McCall's Approach



Quality Metrics: Deriving McCall's Quality Metrics

- Assess a set of quality factors on a scale of 0 (low) to 10 (high)
- Each of McCall's Quality Metrics is a weighted sum of different quality factors
- Weighting is determined by product requirements
- Example:
 - $\text{Correctness} = \text{Completeness} + \text{Consistency} + \text{Traceability}$
 - Completeness is the degree to which full implementation of required function has been achieved
 - Consistency is the use of uniform design and documentation techniques
 - Traceability is the ability to trace program components back to analysis
- This technique depends on good objective evaluators because quality factor scores can be subjective

References

- Robert B. Grady – Practical Software Metrics for Project Management and Process Improvement